

専門コース演習 III FLASK 入門

担当: 石橋 直樹

E-Mail: n-ishi@musashino-u.ac.jp

1 講義の目的

Python の Web フレームワークは色々提案されているが、本講義では計量フレームワークとして広く使われる Flask の使い方を述べる。

2 Flask の導入 (復習)

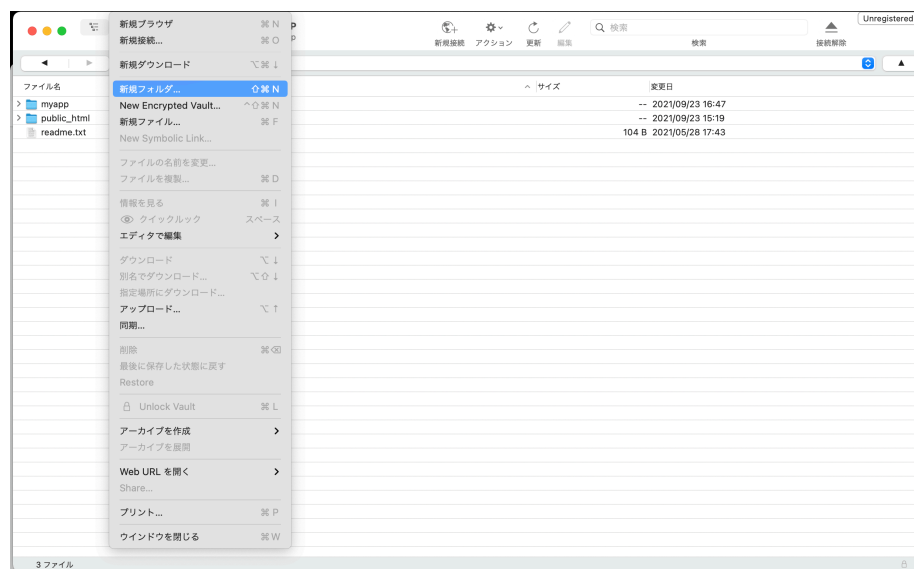
実習に用いるサーバには、計量フレームワーク flask 0.12.2 が導入済みである。

- <https://pypi.org/project/Flask/0.12.2/>
- <https://msiz07-flask-docs-ja.readthedocs.io/ja/latest/index.html>

従って、各自の PC 上で flask を用いて実装、同サーバにデプロイできる。ただし、flask を使う場合、wsgi というインターフェースを使う必要がある。以下に基本的な使い方を示す。

[手順 1] プロジェクトディレクトリの作成

Cyberduck を用いて、実習サーバ内にプロジェクトディレクトリを作成する (例の場合 myapp)。





[手順 2] py ファイルの作成とアップロード

以下のように py ファイルを作成する。なお、この例の場合、名前を myapp.py とする。できあがったファイルを前述の myapp ディレクトリに転送する。

```
1 from flask import Flask
2
3 app = Flask(__name__)
4
5 @app.route("/")
6 def hello_world():
7     return "<p>Hello, World!</p>"
```

[手順 3] wsgi ファイルの作成とアップロード

以下のように wsgi ファイルを作成する。なお、この例の場合、名前を flask.wsgi とする。できあがったファイルを前述の myapp ディレクトリに転送する。

```
1 import sys
2 import os
3 DIR=os.path.dirname(__file__)
4 sys.path.append(DIR)
5 sys.path.insert(0, '/home/s0000001/myapp/')
6 from myapp import app as application
```

[手順 4] 利用申請

以下のような内容の DM を石橋へ送ること。これにより、サーバの設定に上記 wsgi を追記する (手動!)。以下記入例。

```
1 サーバのユーザ名: s0000001
2 プロジェクトディレクトリ: /home/s0000001/myapp/
3 wsgi の場所: /home/s0000001/myapp/flask.wsgi
```

以上の過程を経て構築されたファイルへは、次の URL からアクセスできるよう設定される。CGI 等とは URL が変わるため、注意すること。

- <http://muds.gdl.jp/s0000001/>

3 Flask 版チャット

Flask の特徴は様々だが、導入する最大のメリットはバックエンドとフロントエンドの切り分けが明確化される点にある。これは特に、Flask の render_template メソッドによる。まず、次のように myapp.py を作成する。

```

1 from flask import Flask, request, render_template
2 import psycopg2
3
4 connection = psycopg2.connect("host=localhost dbname=s0000001 user=s0000001 password=
XXXXXXX")
5
6 app = Flask(__name__)
7
8 @app.route("/", methods=['GET', 'POST'])
9 def chat():
10     try:
11         if request.method == 'POST' \
12             and "uname" in request.form and "message" in request.form and len(request.
form['uname'])>0 and len(request.form['message'])>0:
13             uname=request.form['uname']
14             message=request.form['message']
15             cur = connection.cursor()
16             cur.execute("insert into pychat(uname,message,pdate) values('%s','%s',
current_timestamp);" % (uname, message))
17             connection.commit()
18             cur.close()
19     except Exception as e:
20         return str(e)
21
22     cur = connection.cursor()
23     cur.execute("select cid,uname,message,pdate from pychat order by cid desc;")
24     chat = cur.fetchall()
25     return render_template('index.html', title='flask chat', chat=chat)

```

さらに、myapp フォルダ内に templates フォルダを作成、次の内容を layout.html として templates フォルダにアップロードする。

```

1 <html>
2 <head>
3 <title>{{ title }}</title>
4 </head>
5 <body>
6 {% block content %}
7     <!-- コンテンツ -->
8 {% endblock %}
9 </body>
10 </html>

```

さらに次の内容を index.html として作成、同 templates フォルダに転送する。

```

1 {% extends "layout.html" %}
2 {% block content %}
3
4 <form action="." method="POST">
5 ユーザ名: <input type="text" name="uname"><br />
6 message: <input type="text" name="message" size="40"><br />
7 <input type="submit" value="chat!">
8 </form>
9
10 <ul>
11 {% for eachpost in chat %}
12 <li>{{ eachpost[1] }}: {{ eachpost[2] }} ({{ eachpost[3] }})</li>
13 {% endfor %}
14
15 {% endblock %}

```

これらの HTML は単一のファイルにまとめることもできるが、複数のテンプレートで CSS や javascript などを共有する場合、こういった分け方 (デザインテンプレート部と表示部の分離) が有効である。例えば、上記 layout.html に bootstrap 等のモジュール読み込みや共通の CSS、メニューバーなどを記述し、index.html はコンテンツの表示のみのデザインに注力する、などといった作り方が有用である。

4 API 構築

さらに、Flask は URL の処理や JSON 生成などにも便利な機能を有する。従って、Web サービスを開発し使用環境をブラウザ向けに提供すると同時に、API を提供することも可能である。以下に例を示す。

```
1 from flask import Flask, request, render_template, jsonify
2 import psycopg2
3 import psycopg2.extras
4 import json
5
6 connection = psycopg2.connect("host=localhost dbname=s0000001 user=s0000001 password=
   XXXXX")
7
8 app = Flask(__name__)
9
10 @app.route("/", methods=['GET', 'POST'])
11 def chat():
12     try:
13         if request.method == 'POST' and "uname" in request.form and "message" in
            request.form and len(request.form['uname'])>0 and len(request.form['message'])>0:
14             uname=request.form['uname']
15             message=request.form['message']
16             cur = connection.cursor()
17             cur.execute("insert into pychat(uname,message,pdate) values('%s','%s',
                current_timestamp);" % (uname, message))
18             connection.commit()
19             cur.close()
20     except Exception as e:
21         return str(e)
22
23     cur = connection.cursor()
24     cur.execute("select cid,uname,message,pdate from pychat order by cid desc;")
25     chat = cur.fetchall()
26     return render_template('index.html', title='flask chat', chat=chat)
27     cur.close()
28
29 @app.route("/dates")
30 def dates():
31     cur = connection.cursor(cursor_factory=psycopg2.extras.DictCursor)
32     cur.execute("select date(pdate),count(cid) from pychat group by date(pdate) order
        by date(pdate);")
33     res = cur.fetchall()
34     dict_res = []
35     for row in res:
36         dict_res.append(dict(row))
37     return jsonify(dict_res)
38     cur.close()
39 @app.route('/date/<pdate>')
40 def date(pdate):
41     cur = connection.cursor(cursor_factory=psycopg2.extras.DictCursor)
42     cur.execute("select cid,uname,message from pychat where date(pdate)='%s';" % pdate)
43     res = cur.fetchall()
44     dict_res = []
45     for row in res:
46         dict_res.append(dict(row))
47     return jsonify(dict_res)
48     cur.close()
```

これらは次の URL から確認できる。

- <http://muds.gdl.jp/s0000001/>
- <http://muds.gdl.jp/s0000001/dates>
- <http://muds.gdl.jp/s0000001/date/2021-10-6>

5 bootstrap の使用 (外部ファイルの読み込み)

bootstrap 最新版をダウンロードする。

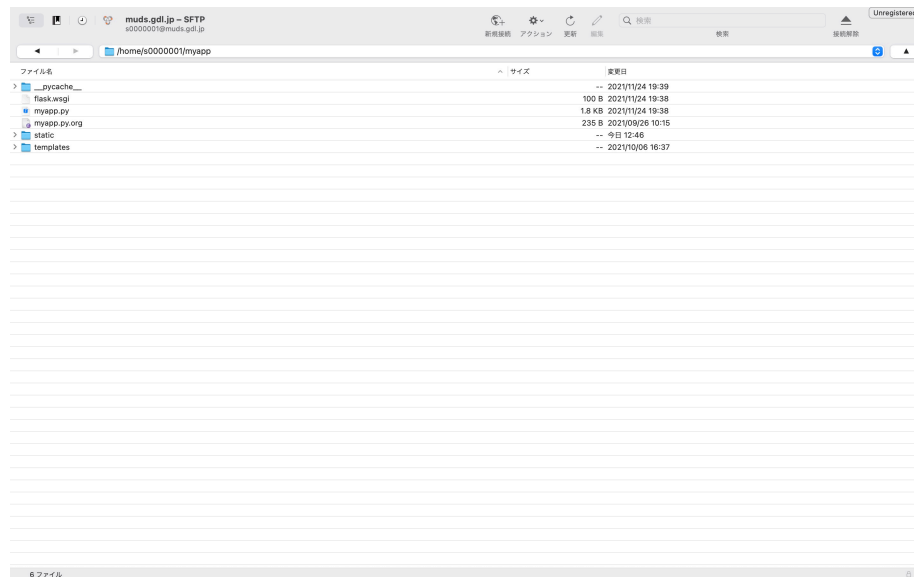
- <https://getbootstrap.jp/>

The image shows two screenshots of the Bootstrap 5 Japanese website. The top screenshot is the homepage, featuring the Bootstrap logo and a large 'B' icon. It highlights the 'ダウンロード' (Download) button, which is highlighted with a red box. Below the main heading, there is a section for 'Installation' with a code snippet: `$ npm install bootstrap@next`. The bottom screenshot shows the 'ダウンロード' (Download) page, which provides detailed instructions on how to download and install Bootstrap. It includes a 'Download' button, which is also highlighted with a red box, and a list of links for various resources like CSS, JavaScript, and source code.

解凍すると、css フォルダ、js フォルダが入っている。これらのファイルを、flask フォルダ内に作成した static フォルダに入れる。

今日		1か月以上前	
02flask.tex	●	flask.wsgi	●
		myapp.py	●
1か月以上前		static	● >
02flask.aux	●	templates	● >
02flask.dvi	●		
02flask.log	●		
02flask.pdf	●		
02flask.tex~	●		
images	● >		
myapp	● >		

static フォルダは、サーバ内の flask フォルダにも転送する。



テンプレートフォルダ内に次のように記載すると、これらのファイルを読み込むことができる。

5.1 layout.html

```

1 <html>
2 <head>
3 <title>{{ title }}</title>
4 <link rel="stylesheet" href="{ url_for("static", filename="css/bootstrap.min.css")
  }}">
5 </head>
6 <body>
7 {% block content %}
8 <!-- ここにメインコンテンツを書く -->
9 {% endblock %}
10 <script src="{ url_for("static", filename="js/bootstrap.min.js") }}"></script>
11 </body>
12 </html>

```

5.2 index.html

```
1 {% extends "layout.html" %}
2 {% block content %}
3
4 <nav class="navbar navbar-expand-lg bg-body-tertiary">
5   <div class="container-fluid">
6     <a class="navbar-brand" href="/">pychat</a>
7     <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
      target="#navbarSupportedContent" aria-controls="navbarSupportedContent" aria-
      expanded="false" aria-label="Toggle navigation">
8       <span class="navbar-toggler-icon"></span>
9     </button>
10    <div class="collapse navbar-collapse" id="navbarSupportedContent">
11      <form class="d-flex" role="post" action="/" method="POST">
12        <input class="form-control me-2" type="post" placeholder="ユーザー名" aria-label="
          UserName" name="uname">
13        <input class="form-control me-2" type="post" placeholder="メッセージ" aria-label
          ="Message" name="message">
14        <button class="btn btn-outline-success" type="submit">chat!</button>
15      </form>
16    </div>
17  </div>
18 </nav>
19
20
21 <ul class="list-group">
22 {% for eachpost in chat %}
23   <li class="list-group-item">{{ eachpost[1] }}: {{ eachpost[2] }} ({{ eachpost[3]
      }})</li>
24 {% endfor %}
25
26 {% endblock %}
```
